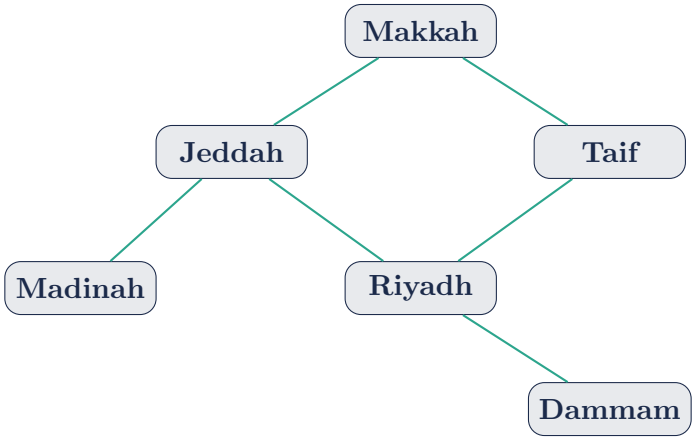
CS3081 – Artificial Intelligence • Spring 2026

Lab 3

Search & Knowledge



Instructor:	**Dr. Naila Marir**
Duration:	**2 Hours**
Tools:	**VS Code + Python 3**
Files:	**part1_search.py** **part2_knowledge.py** **logic.py**
Student Name:	_____
Student ID:	_____

1. Lab Overview

What You Will Build

In this lab you will build two small Python programs:

- **Part 1 – Search:** A route finder that uses **BFS (Breadth-First Search)** to travel between Saudi cities.
- **Part 2 – Knowledge:** A logic detective that uses a **Knowledge Base** and **model checking** to deduce who is guilty.

Before You Start – VS Code Checklist

Open VS Code. Then open a terminal inside it (`Terminal > New Terminal`). Run the checks below:

1. Check Python is installed: `python --version` (should show 3.x)
2. Create a new folder called `Lab3` somewhere on your computer.
3. Copy **all three files** into that folder: `part1_search.py`, `part2_knowledge.py`, `logic.py`
4. In VS Code: **File** → **Open Folder** → select your `Lab3` folder.

Lab Files Summary:

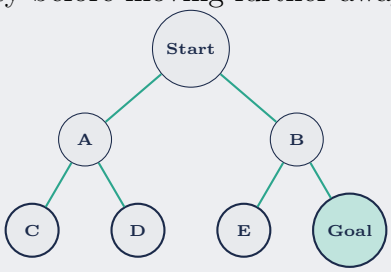
File	Description
`part1_search.py`	Your work for Part 1 – city graph and BFS
`part2_knowledge.py`	Your work for Part 2 – knowledge base and deduction
`logic.py`	Helper classes for Part 2 – **do not edit this file**

2. Part 1 – Search: Finding a Route with BFS

2.1. Background: What is BFS?

Breadth-First Search (BFS)

BFS is a **search algorithm** that explores a graph level by level – it visits all neighbors of the current city before moving further away.



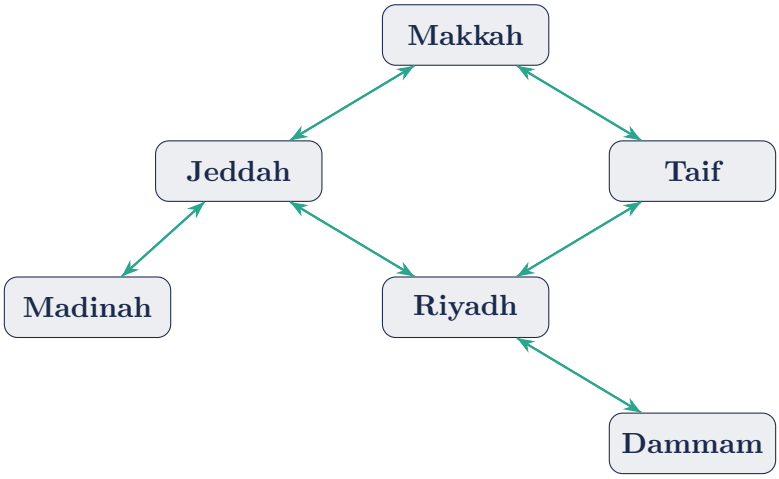
BFS uses a **queue** (FIFO – First In, First Out).

- Add the start city to the queue.
- Take the *front* city, check its neighbors, add them to the *back*.
- Repeat until you reach the goal.

BFS always finds the shortest path (fewest hops).

2.2. The City Map

Our map has 6 Saudi cities. Each city is connected to one or more neighbors.



In Python, this map is stored as a **dictionary**:

```
city_map = {
    "Makkah":   ["Jeddah", "Taif"],
    "Jeddah":   ["Makkah", "Riyadh", "Madinah"],
    "Taif":     ["Makkah", "Riyadh"],
    "Riyadh":   ["Jeddah", "Taif", "Dammam"],
```

```
    "Madinah": ["Jeddah"],
    "Dammam":  ["Riyadh"]
}
```

Listing 1: City map as a Python dictionary (part1_search.py lines 14–21)

Reading a Python Dictionary

`"Jeddah": ["Makkah", "Riyadh", "Madinah"]` means: *from Jeddah you can reach Makkah, Riyadh, and Madinah.*

2.3. Step-by-Step Code Walkthrough

Open `part1_search.py` in VS Code. Read each section carefully.

Step 1 – The Node Class (lines 28–40)

A **Node** represents a city we are currently considering during the search.

```
class Node:
    def __init__(self, city, parent=None):
        self.city = city        # Where we are now
        self.parent = parent    # Which node came before us
```

The `parent` pointer lets us trace the full path back to the start once we reach the goal. The `path()` method does this tracing.

Example: If the path is Makkah → Jeddah → Riyadh, then:

- Riyadh node has parent = Jeddah node
- Jeddah node has parent = Makkah node
- Makkah node has parent = `None`

Step 2 – The Queue and Frontier (lines 50–65)

BFS uses a **queue** so we always process cities in the order we discovered them.

```
from collections import deque    # deque = double-ended queue

frontier = deque()
frontier.append(start_node)      # Add start to the back
explored = set()                 # Empty set -- nothing explored yet
```

Operation	Code	Meaning
Add to back	`frontier.append(node)`	Discover a new city
Remove from front	`frontier.popleft()`	Process the next city
Check explored	`if city not in explored`	Avoid revisiting

Step 3 – The BFS Loop (lines 67–80)

The main loop keeps running until either the goal is found or the frontier is empty.

```
while frontier:
    current_node = frontier.popleft()       # Take the FIRST city
```

```python
    current_city = current_node.city

    if current_city == goal:                # Did we arrive?
        return current_node.path()          # Trace and return the path

    explored.add(current_city)              # Mark as explored

    for neighbor in city_map[current_city]: # Check all neighbors
        if neighbor not in explored:
            neighbor_node = Node(city=neighbor, parent=current_node)
            frontier.append(neighbor_node)
            explored.add(neighbor)
```

2.4. Run the Program

In the VS Code terminal, make sure you are inside the `Lab3` folder:

```
cd Lab3
python part1_search.py
```

You should see output like this:

```
Searching from 'Makkah' to 'Dammam' using BFS...

  Step 1: Visiting 'Makkah'
  Step 2: Visiting 'Jeddah'
  Step 3: Visiting 'Taif'
  Step 4: Visiting 'Riyadh'
  Step 5: Visiting 'Madinah'
  Step 6: Visiting 'Dammam'

Goal reached! 'Dammam' found.

Path found (4 steps):
Makkah -> Jeddah -> Riyadh -> Dammam
```

Understanding the Output

- Each "Step" is BFS visiting one city from the frontier.
- Notice that BFS explored Madinah (a dead-end branch) before finding Dammam. This is because BFS explores level by level.
- The final path **Makkah** → **Jeddah** → **Riyadh** → **Dammam** is the shortest route (4 hops).

2.5. Part 1 Exercises

Exercise 1 – Change the Goal City

At the bottom of `part1_search.py` (line 91–92), change the destination:

```
start_city = "Makkah"
goal_city  = "Madinah"     # <-- change this
```

Run the program again. Answer these questions in your lab report:

a. How many steps did BFS take to reach Madinah?
b. What is the path it found?
c. Is this the shortest path? Verify by looking at the city map diagram.

Exercise 2 – Add a New City

Add a new city **Abha** to the map, connected to Taif:

a. In `city_map`, add: `"Abha": ["Taif"]`
b. Also update Taif's list to include Abha.
c. Set `goal_city = "Abha"` and run.
d. Write the path BFS found from Makkah to Abha.

3. Part 2 – Knowledge: Logic Detective

3.1. Background: Propositional Logic and the Knowledge Base

Key Concepts from Lecture 3

- **Propositional Symbol:** A statement that is either **True** or **False**. *e.g., "It is raining"*
- **Knowledge Base (KB):** A set of sentences we *know* are true.
- **Entailment ($\models$):** KB $\models \alpha$ means: in every world where KB is true, α is also true.
- **Model Checking:** We check all possible True/False combinations. If α is true whenever the KB is true – it is entailed.

Logical Connectives – Quick Reminder

Symbol	Name	Example
`Not(P)`	Negation (NOT)	Sara does **not** have a key
`And(P, Q)`	Conjunction (AND)	Raining AND cold
`Or(P, Q)`	Disjunction (OR)	Lina OR Sara has a key
`Implication(P, Q)`	Implication (IF→)	IF key THEN guilty

3.2. The Mystery Scenario

The Case: A valuable item went missing at Effat University.

Three suspects: **Sara**, **Lina**, **Nora**

The Clues (what our AI knows):

Clue 1: If Lina has a key → Lina is guilty.

Clue 2: Sara does *not* have a key.

Clue 3: Lina was seen near the room. (this is a known fact)

Clue 4: If Lina was seen near the room → Lina has a key.

The Question: Can our AI prove that Lina is guilty?

3.3. Step-by-Step Code Walkthrough

Open `part2_knowledge.py` in VS Code. Read each section carefully.

Step 1 – Define the Symbols (lines 22–29)

Each `Symbol` represents a fact that can be **True** or **False**.

```
from logic import Symbol, Not, And, Or, Implication, KB,
    check_all

sara_key     = Symbol("SaraHasKey")
lina_key     = Symbol("LinaHasKey")
lina_seen    = Symbol("LinaSeenNearRoom")
lina_guilty  = Symbol("LinaIsGuilty")

# All symbol NAMES -- needed for model checking
all_symbols = ["SaraHasKey", "LinaHasKey", "
    LinaSeenNearRoom", "LinaIsGuilty"]
```

What is a Symbol?

A `Symbol` is just a named placeholder. It has no truth value by itself – the truth value is determined when we assign a model (a True/False table). This is exactly what we learned in the lecture truth table examples.

Step 2 – Build the Knowledge Base (lines 33–45)

We add each clue using `kb.tell(...)`. The KB stores all the things we know.

```
kb = KB()

# Clue 1: IF lina_key THEN lina_guilty
kb.tell(Implication(lina_key, lina_guilty))

# Clue 2: NOT sara_key
kb.tell(Not(sara_key))

# Clue 3: lina_seen is TRUE (a fact we directly know)
kb.tell(lina_seen)

# Clue 4: IF lina_seen THEN lina_key
kb.tell(Implication(lina_seen, lina_key))
```

Reading Implication in Python

`Implication(A, B)` means "IF A is True THEN B must be True".
Compare to the lecture: $A \rightarrow B$ is false only when A is **True** and B is **False**.

Step 3 – Ask the Knowledge Base (lines 49–61)

We use `check_all` to perform model checking – it tries all True/False combinations and checks whether our query is always true when the KB is satisfied.

```
answer = check_all(kb, lina_guilty, all_symbols)

if answer:
    print("YES -- The KB ENTAILS that Lina is guilty.")
else:
    print("NO  -- The KB does NOT entail that Lina is
        guilty.")
```

`check_all` takes three arguments:

- `kb` – the knowledge base with all our clues
- `lina_guilty` – the query we want to prove
- `all_symbols` – the list of symbol names (used to generate all models)

3.4. How Model Checking Works Internally

The `check_all` function in `logic.py` tries **all** possible worlds. With 4 symbols, there are $2^4 = 16$ possible worlds (combinations of True/False).

SaraKey	LinaKey	LinaSeen	LinaGuilty	KB True?	Query True?
False	**False**	**False**	**False**	**False**	(skip)
False	**False**	**True**	**False**	**False**	(skip)
...	...	...	...	**False**	(skip)
False	**True**	**True**	**True**	**True**	**True** ✓

Only one model makes the KB true. In that model, `LinaIsGuilty` is also **True**. Therefore: **KB** $\models$ **LinaIsGuilty** – the KB *entails* Lina is guilty.

3.5. Run the Program

In the VS Code terminal (make sure you are in the `Lab3` folder):

```
python part2_knowledge.py
```

Expected output:

```
==================================================
  CS3081 Lab 3 -- Knowledge Base Detective
==================================================

Query:  Is Lina guilty?
YES  -- The KB ENTAILS that Lina is guilty.

Query:  Does Sara have a key?
NO   -- The KB does NOT entail Sara has a key.
```

Why does Sara having a key return NO?

Our KB says `Not(sara_key)` – meaning we *know* Sara does *not* have a key. So asking "does Sara have a key?" correctly returns **False**. The KB proves she does not have one!

3.6. Part 2 Exercises

Exercise 3 – Add a New Clue

Add the following new clue to the KB (after the existing `kb.tell` lines):

```
nora_key = Symbol("NoraHasKey")
kb.tell(nora_key)  # Nora has a key (new evidence)
```

Also add `"NoraHasKey"` to `all_symbols`.

Then add a new query to find out if Nora is guilty:

```
nora_guilty = Symbol("NoraIsGuilty")
answer3 = check_all(kb, nora_guilty, all_symbols)
print(f"Is Nora guilty? {answer3}")
```

Run the program and answer:

a. Does the KB entail Nora is guilty? Why or why not?
b. What extra clue would you need to add to prove Nora is guilty? Write the `kb.tell` line.

Exercise 4 – Remove a Clue

Try commenting out (adding # at the start of) **Clue 3** (`kb.tell(lina_seen)`):

```
# kb.tell(lina_seen)    <-- commented out
```

Run the program again and answer:

a. Is Lina still guilty according to the KB? What changed?
b. Explain in your own words why removing that one clue changed the result.

4. Discussion Questions

Answer these questions in your lab report. Full sentences, please.

Search Questions

1. In Part 1, BFS visited **Madinah** even though it is not on the path to Dammam. Why does BFS visit "wrong" cities, and is this a problem?

2. Compare BFS to DFS (from Lab 1). For finding the *shortest* path, which is better and why?

3. Imagine you added **road distances** (e.g., Jeddah to Riyadh = 900 km). BFS does not consider distances – it just counts hops. What kind of search algorithm could use those distances to find a *cheaper* route?

Knowledge Questions

4. In your own words, what is the difference between **implication** ($P \rightarrow Q$) and **entailment** ($\text{KB} \models \alpha$)?

5. In Part 2, model checking tried all $2^4 = 16$ worlds. If we had 10 symbols instead of 4, how many worlds would need to be checked? Is this practical for very large problems?

5. What to Submit

Submit **one ZIP file** named `Lab3_YourName_ID.zip` containing:

Item	Details
`part1_search.py`	Your modified file with Exercises 1 & 2 completed
`part2_knowledge.py`	Your modified file with Exercises 3 & 4 completed
Screenshots	Terminal output for both parts (all exercises)
Lab Report (PDF or Word)	Answers to all discussion questions

Screenshot Tip

On Windows: press **Win + Shift + S** to take a region screenshot.
Paste it directly into your Word report or save as an image file.

Grading

Component	Marks
Part 1 – Exercise 1 (change goal, observations)	15
Part 1 – Exercise 2 (add new city)	15
Part 2 – Exercise 3 (add new clue)	20
Part 2 – Exercise 4 (remove a clue)	20
Discussion Questions (5 questions)	30
Total	**100**

Good luck! Think like an AI.

Dr. Naila Marir

CS3081 • Artificial Intelligence • Spring 2026